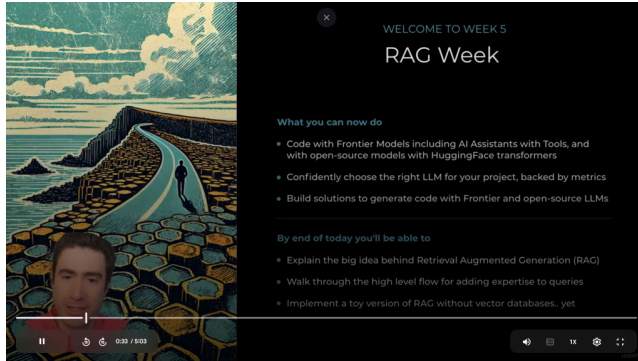


Day 1

31 Tháng Ba 2025 4:40 CH



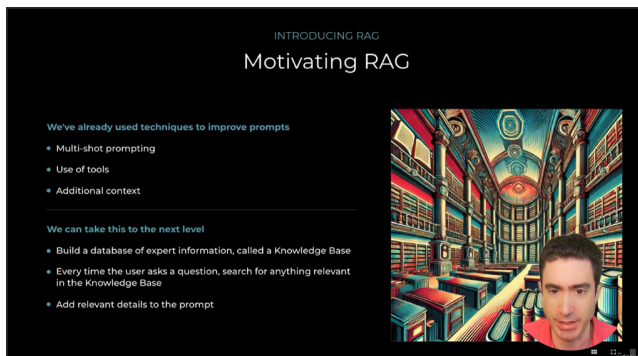
Chủ đề tuần: "RAG Week" – tuần học về RAG.

Mục tiêu:

- Lập trình với các mô hình Frontier và các mô hình mã nguồn mở từ HuggingFace.
- Lựa chọn mô hình LLM phù hợp dựa trên các tiêu chí đánh giá.
- Xây dựng giải pháp tạo mã với Frontier và LLM mã nguồn mở.

Kết quả mong đợi trong ngày:

- Giải thích ý tưởng chính của RAG.
- Hiểu quy trình tổng quan để bổ sung kiến thức vào truy vấn.
- Triển khai phiên bản thử nghiệm của RAG mà chưa cần đến cơ sở dữ liệu vector.



Các kỹ thuật cải thiện prompts:

- Multi-shot prompting: Sử dụng nhiều ví dụ trong prompt để hướng dẫn mô hình.
- Use of tools: Sử dụng các công cụ hỗ trợ để hỗ trợ quá trình tạo sinh văn bản.
- Additional context: Thêm ngữ cảnh bổ sung vào prompt để cung cấp thêm thông tin cho mô hình.

Nâng cao hệ thống RAG:

- Xây dựng cơ sở dữ liệu thông tin chuyên gia (Knowledge Base).
- Tìm kiếm thông tin liên quan trong Knowledge Base mỗi khi người dùng đặt câu hỏi.
- Thêm thông tin liên quan vào prompt.



- **RAG là gì?** RAG là một kỹ thuật cải thiện phản hồi của mô hình ngôn ngữ lớn (LLM) bằng cách cung cấp thêm thông tin ngữ cảnh từ một cơ sở tri thức (Knowledge Base).
- **Quy trình hoạt động:**
 1. Người dùng đặt câu hỏi thông qua giao diện chat.
 2. Câu hỏi được chuyển đến mã code.
 3. Thông thường, câu hỏi sẽ được chuyển trực tiếp đến LLM.
 4. Nhưng với RAG, trước khi chuyển đến LLM, mã code sẽ truy vấn cơ sở tri thức để tìm kiếm thông tin liên quan đến câu hỏi.
 5. Nếu tìm thấy, thông tin liên quan sẽ được trích xuất và thêm vào prompt (lời nhắc) gửi đến LLM.
 6. LLM tạo ra phản hồi dựa trên prompt đã được bổ sung thông tin.
 7. Phản hồi được gửi lại cho người dùng.
- **Mục đích:** Cung cấp cho LLM ngữ cảnh bổ sung để tạo ra phản hồi chính xác và phù hợp hơn với câu hỏi của người dùng.

INTRODUCING RAG

A small example of the small idea

Business setup

- We work for an Insurance Tech startup
- We have a Knowledge Base of the company shared drive
- Task is to build an AI Knowledge Worker

The Blunt Instrument Implementation

- Read names of products and employees
- See if questions refer to employee or products by name
- Add relevant details to the prompt



Ứng dụng thực tế của RAG:

- Xây dựng một "AI Knowledge Worker" cho một công ty khởi nghiệp công nghệ bảo hiểm hư cấu tên là "Insurer-LLM".
- Sử dụng cơ sở tri thức (Knowledge Base) từ thư mục chia sẻ của công ty.
- Mục tiêu là tạo ra một hệ thống AI có khả năng trả lời các câu hỏi về thông tin công ty.

Triển khai RAG đơn giản (Blunt Instrument Implementation):

- Đọc các tệp thông tin về sản phẩm và nhân viên từ cơ sở tri thức.
- Lưu trữ thông tin này trong một cấu trúc dữ liệu (ví dụ: từ điển).
- Khi nhận được câu hỏi, kiểm tra xem tên nhân viên hoặc sản phẩm có xuất hiện trong câu hỏi hay không.
- Nếu có, chèn toàn bộ thông tin liên quan vào prompt gửi đến LLM.
- Đây là một cách triển khai thủ công, "brute force" của RAG, nhưng nó minh họa nguyên tắc cơ bản.

Lợi ích của RAG:

- Cải thiện hiệu suất của mô hình ngôn ngữ bằng cách cung cấp ngữ cảnh bổ sung.
- Cho thấy rằng RAG không phải là "phép màu" mà là một kỹ thuật thực tế.

Các bước tiếp theo:

- Sau khi minh họa triển khai đơn giản, sẽ chuyển sang các phương pháp RAG phức tạp và hiệu quả hơn.
- Sử dụng JupyterLab để xây dựng RAG tự chế.

day1.ipynb

File Edit View Run Kernel Tools Settings Help

day1.ipynb

Python 3 (ipykernel)

Filter files by name

Name

Modified

week5/

knowledge-base

2d ago

vector_db

2d ago

day1.ipynb

2d ago

day2.ipynb

17h ago

day3.ipynb

2d ago

day4.ipynb

2d ago

day5.ipynb

2d ago

Expert Knowledge Worker

A question answering agent that is an expert knowledge worker

To be used by employees of Insurelin, an Insurance Tech company

The agent needs to be accurate and the solution should be low cost.

This project will use RAG (Retrieval Augmented Generation) to ensure our question/answering assistant has high accuracy.

This first implementation will use a simple, brute-force type of RAG.

```
1 # Imports
2 import os
3 import sys
4 from vector_store import load_vector
5 import gradio as gr
6 from agent import Agent
7
8 # If price is a factor for our company, so we're going to use a low cost model
9 MODEL = "gpt-4o-mini"
10
11 # Load environment variables in a file called .env
12 load_dotenv()
13 os.environ["OPENAI_API_KEY"] = os.getenv("OPENAI_API_KEY", "your-key-if-not-using-env")
14 os.environ["SERPAPI_KEY"] = os.getenv("SERPAPI_KEY", "your-key-if-not-using-env")
15
16 context = {}
17
18 employees = get_all_from("knowledge-base/employees/*")
```

Môi trường làm việc:

- Sử dụng JupyterLab để xây dựng một hệ thống RAG (Retrieval-Augmented Generation) tự chế.
- Làm việc trong thư mục "week5" và sử dụng notebook "day1.ipynb".

Dữ liệu:

- Công ty hư cấu "Insurer-LLM", một công ty công nghệ bảo hiểm.
- Cơ sở tri thức (Knowledge Base) là thư mục "knowledge-base" chứa dữ liệu của công ty.
- Thư mục "knowledge-base" bao gồm bốn thư mục con: "company", "contracts", "employees", và "products".
- Dữ liệu trong các thư mục này được tạo ra bởi LLM (GPT-4 hoặc Claude), với một số chỉnh sửa thủ công.
- Dữ liệu bao gồm các tài liệu markdown (.md) chứa thông tin về công ty, hợp đồng, nhân viên và sản phẩm.

Mục tiêu:

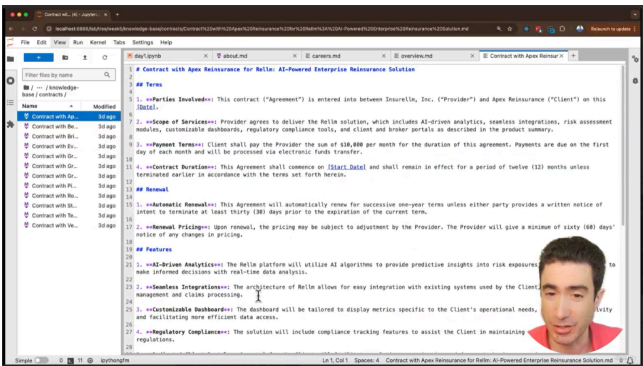
- Xây dựng một hệ thống RAG đơn giản để trả lời các câu hỏi về thông tin công ty.
- Hiểu rõ hơn về cách RAG hoạt động bằng cách triển khai một phiên bản tự chế.

Quy trình:

- Sử dụng dữ liệu từ thư mục "knowledge-base" để cung cấp ngữ cảnh cho LLM.
- Triển khai một phương pháp "brute-force" (đơn giản, trực tiếp) để truy xuất và sử dụng thông tin từ cơ sở tri thức.

Thông tin bổ sung:

- Dữ liệu được tạo ra một cách giả định để minh họa quá trình.
- Mục tiêu là xây dựng một "expert knowledge worker" (người làm việc tri thức chuyên gia) có khả năng trả lời các câu hỏi về công ty.



Dữ liệu trong Knowledge Base:

- Thư mục "company":** Chứa thông tin tổng quan về công ty Insurer-LLM, bao gồm lịch sử thành lập (năm 2015 bởi Avery Lancaster) và các thông tin cơ bản khác.
- Thư mục "contracts":** Chứa các hợp đồng của công ty, bao gồm các điều khoản, điều kiện gia hạn và các tính năng được bao gồm trong hợp đồng.
- Thư mục "employees":** Chứa hồ sơ nhân sự của các nhân viên, bao gồm cả CEO Avery Lancaster.
- Thư mục "products":** Chứa mô tả về các sản phẩm của Insurer-LLM, bao gồm Car-LLM (bảo hiểm ô tô) và Home-LLM (bảo hiểm nhà).

Định dạng dữ liệu:

- Các tài liệu trong Knowledge Base được viết bằng định dạng Markdown (.md).

Nguồn gốc dữ liệu:

- Dữ liệu được tạo ra bởi các mô hình ngôn ngữ lớn (Frontier Models), có vẻ như là GPT-4 hoặc Claude.

Mục tiêu:

- Sử dụng dữ liệu từ Knowledge Base để xây dựng một hệ thống RAG có khả năng trả lời các câu hỏi về thông tin công ty, hợp đồng, nhân viên và sản phẩm.
- Hiểu rõ hơn về cấu trúc và nội dung của dữ liệu được sử dụng trong hệ thống RAG.

Thông tin bổ sung:

- Tên "Avery Lancaster" sẽ xuất hiện nhiều lần trong quá trình xây dựng hệ thống RAG.
- Các hợp đồng chứa thông tin chi tiết về các tính năng quan trọng sẽ được sử dụng sau này.
- Các sản phẩm được mô tả chi tiết, bao gồm cả lộ trình phát triển.

Mục tiêu: Xây dựng một hệ thống có khả năng trả lời câu hỏi về công ty bằng cách chèn ngữ cảnh vào prompt một cách thủ công.

Mô hình ngôn ngữ sử dụng: GPT-4-mini (một phiên bản nhỏ hơn của GPT-4).

Xử lý dữ liệu nhân viên:

- Đọc danh sách các tệp nhân viên từ thư mục knowledge-base/employees/.
- Trích xuất tên nhân viên (họ) từ tên tệp.
- Đọc nội dung của từng tệp nhân viên và lưu trữ vào một từ điển (dictionary) có tên là context, với key là tên nhân viên (họ) và value là nội dung tệp.
- Kiểm tra nội dung của từ điển context cho nhân viên Lancaster (CEO).

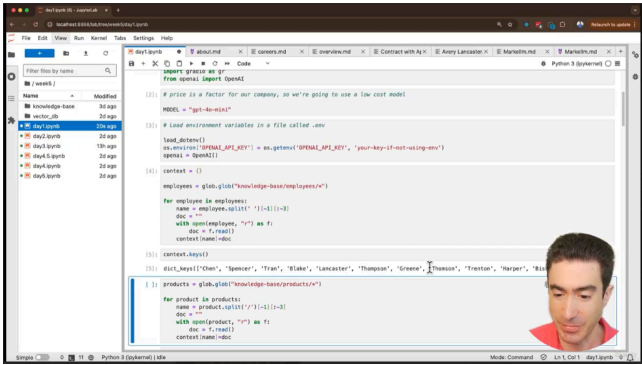
Xử lý dữ liệu sản phẩm:

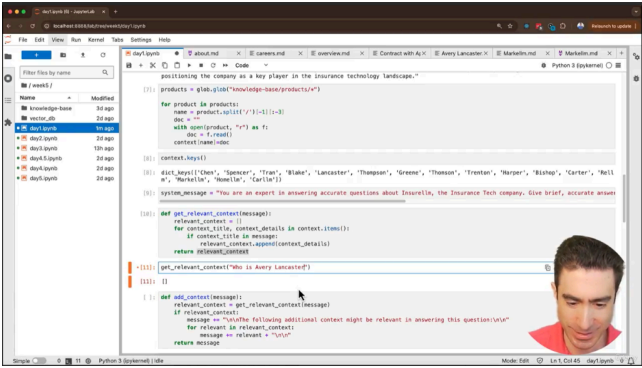
- Tương tự như xử lý dữ liệu nhân viên, đọc danh sách các tệp sản phẩm từ thư mục knowledge-base/products/.
- Trích xuất tên sản phẩm từ tên tệp.
- Đọc nội dung của từng tệp sản phẩm và lưu trữ vào cùng từ điển context.
- Kiểm tra các key trong từ điển context để xác nhận rằng nó chứa cả tên nhân viên (họ) và tên sản phẩm.

Nguyên tắc cơ bản của RAG (Retrieval-Augmented Generation) được minh họa:

- Đọc dữ liệu từ cơ sở tri thức (Knowledge Base) và lưu trữ vào một cấu trúc dữ liệu (từ điển context).
- Sử dụng cấu trúc dữ liệu này để tìm kiếm và chèn ngữ cảnh vào prompt khi đặt câu hỏi.

Lưu ý: Đoạn code được mô tả là "janky code" và "hacked together", cho thấy đây là một cách triển khai đơn giản, không tối ưu.





Xây dựng hệ thống trả lời câu hỏi dựa trên ngữ cảnh:

- Tạo một hệ thống có khả năng trả lời các câu hỏi về công ty "Insurer-LLM" bằng cách sử dụng thông tin từ cơ sở tri thức (Knowledge Base).
- Hệ thống sẽ tìm kiếm thông tin liên quan trong cơ sở tri thức dựa trên câu hỏi của người dùng và cung cấp câu trả lời chính xác.

Sử dụng System Prompt để kiểm soát hành vi của mô hình ngôn ngữ:

- Thiết lập một System Prompt với các hướng dẫn cụ thể để mô hình ngôn ngữ không bịa đặt thông tin và chỉ cung cấp câu trả lời dựa trên ngữ cảnh được cung cấp.
- Ví dụ: "Bạn là một chuyên gia trả lời các câu hỏi chính xác về Insurer-LLM. Hãy đưa ra câu trả lời ngắn gọn, chính xác. Nếu bạn không biết câu trả lời, hãy nói như vậy. Đừng bịa đặt bất cứ điều gì. Nếu bạn không được cung cấp ngữ cảnh liên quan."
- Học được cách sử dụng System Prompt để tăng độ chính xác và giảm thiểu hiện tượng "ảo giác" (hallucination) của mô hình ngôn ngữ.

Triển khai hàm get_relevant_context:

- Xây dựng một hàm có khả năng trích xuất ngữ cảnh liên quan từ cơ sở tri thức dựa trên câu hỏi của người dùng.
- Hàm này sẽ tìm kiếm các từ khóa (tên nhân viên, tên sản phẩm) trong câu hỏi và trả về danh sách các tài liệu liên quan từ cơ sở tri thức.

Hiểu được tính "brittle" (dễ vỡ) của phương pháp đơn giản:

- Phương pháp trích xuất ngữ cảnh được triển khai trong ví dụ này rất đơn giản và dễ bị lỗi.
- Nó chỉ hoạt động khi câu hỏi chứa chính xác tên nhân viên hoặc sản phẩm (bao gồm cả việc viết hoa đúng cách).
- Bạn sẽ học được rằng phương pháp này không đủ mạnh mẽ để xử lý các câu hỏi phức tạp hoặc có lỗi chính tả.

Mình họa nguyên tắc cơ bản của RAG (Retrieval-Augmented Generation):

- Đoạn trích này minh họa cách RAG hoạt động bằng cách trích xuất thông tin liên quan từ cơ sở tri thức và sử dụng nó để trả lời câu hỏi.
- Tuy nhiên, nó cũng cho thấy những hạn chế của một cách triển khai đơn giản và cần có các phương pháp phức tạp hơn để đạt được kết quả tốt hơn.

Xây dựng hàm add_context:

- Tạo một hàm để thêm ngữ cảnh liên quan vào tin nhắn (message) trước khi gửi đến mô hình ngôn ngữ.
- Hàm này sử dụng hàm get_relevant_context đã được xây dựng trước đó để tìm ngữ cảnh liên quan.
- Nếu tìm thấy ngữ cảnh, nó sẽ thêm một đoạn văn bản vào tin nhắn, thông báo rằng ngữ cảnh được thêm vào có thể liên quan đến câu hỏi.

Mình họa tính "brittle" (dễ vỡ) của phương pháp tìm kiếm ngữ cảnh đơn giản:

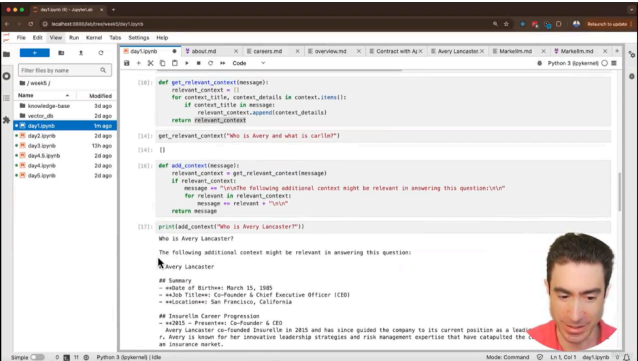
- Ví dụ với "Alex Lancaster" cho thấy rằng hàm chỉ tìm kiếm các từ khóa đơn giản (ví dụ: "Lancaster") mà không phân biệt ngữ cảnh hoặc tên đầy đủ.
- Điều này dẫn đến việc thêm ngữ cảnh không chính xác vào tin nhắn.

Xây dựng hàm chat cho giao diện người dùng Gradio:

- Sử dụng Gradio để tạo một giao diện trò chuyện đơn giản.
- Hàm chat xử lý tin nhắn của người dùng và lịch sử trò chuyện.
- Chuyển đổi lịch sử trò chuyện sang định dạng mà OpenAI API yêu cầu.
- Gọi hàm add_context để thêm ngữ cảnh vào tin nhắn trước khi gửi đến OpenAI API.
- Sử dụng OpenAI API để tạo phản hồi từ mô hình ngôn ngữ.
- Truyền phản hồi trở lại giao diện Gradio.

Hiểu được quy trình cơ bản của ứng dụng trò chuyện sử dụng RAG:

- Nhận tin nhắn từ người dùng.
- Tìm kiếm ngữ cảnh liên quan từ cơ sở tri thức.
- Thêm ngữ cảnh vào tin nhắn.
- Gửi tin nhắn đến mô hình ngôn ngữ.
- Nhận phản hồi từ mô hình ngôn ngữ.
- Hiển thị phản hồi cho người dùng.



For those who haven't encountered Vectors

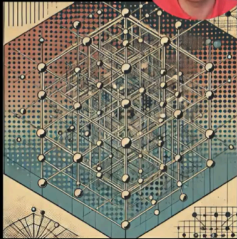
Encoding LLMs and Vector Embeddings

Auto-Encoding vs Auto-Regressive LLMs

- Auto-regressive LLMs predict a future token from the past
- Auto-encoding LLMs produce output based on the full input

Auto-encoding LLMs

- Applications include Sentiment Analysis and classification
- Also used to calculate "Vector Embeddings", representing an input as a list of numbers - i.e. a vector
- Examples include BERT from Google and OpenAI/Embeddings from OpenAI





Khái niệm về Vectors và Vector Embeddings:

- Vectors là một khía cạnh quan trọng của RAG (Retrieval-Augmented Generation).
- Vector embeddings là cách biểu diễn văn bản (thường là câu) dưới dạng một chuỗi số, phản ánh ý nghĩa của văn bản đó.

Sự khác biệt giữa Auto-regressive LLMs và Auto-encoding LLMs:

- Auto-regressive LLMs:** Dự đoán token tiếp theo trong chuỗi dựa trên các token trước đó (ví dụ: GPT-4, Claude, Gemini).
- Auto-encoding LLMs:** Tạo ra đầu ra dựa trên toàn bộ đầu vào (ví dụ: BERT, OpenAI embeddings).

Ứng dụng của Auto-encoding LLMs:

- Phân tích cảm xúc (Sentiment Analysis).
- Phân loại văn bản (Classification).
- Tạo vector embeddings.

Vector Embeddings và ý nghĩa của văn bản:

- Vector embeddings biểu diễn văn bản dưới dạng một điểm trong không gian đa chiều (thường là hàng trăm hoặc hàng nghìn chiều).
- Điểm này phản ánh ý nghĩa của văn bản.

Ví dụ về Auto-encoding LLMs:

- BERT (Google).
- OpenAI embeddings (OpenAI) - mô hình sẽ được sử dụng trong các dự án RAG của khóa học.

Ý nghĩa của "ý nghĩa" trong ngữ cảnh vector embeddings:

- Đoạn trích sẽ giải thích chi tiết hơn về ý nghĩa của việc "phản ánh ý nghĩa" của văn bản thông qua vector embeddings.

These vectors mathematically represent the 'meaning' of an input



Can represent a character, a token, a word, an entire document, or something abstract



Typically have hundreds, or thousands of dimensions



Represent an 'understanding' of the inputs; similar inputs are close to each other



Support 'vector math' like the famous example: "King - Man + Woman = Queen"



Vector embeddings biểu diễn "ý nghĩa" của đầu vào:

- Vector embeddings là một cách biểu diễn toán học cho "ý nghĩa" của dữ liệu đầu vào.
- Dữ liệu đầu vào có thể là một ký tự, token, từ, câu, đoạn văn, tài liệu hoặc thậm chí một khái niệm trừu tượng.

Đặc điểm của vector embeddings:

- Thường có hàng trăm hoặc hàng nghìn chiều (số).
- Các đầu vào có ý nghĩa tương tự sẽ được biểu diễn bằng các vector gần nhau trong không gian vector.

"Ý nghĩa" trong ngữ cảnh vector embeddings:

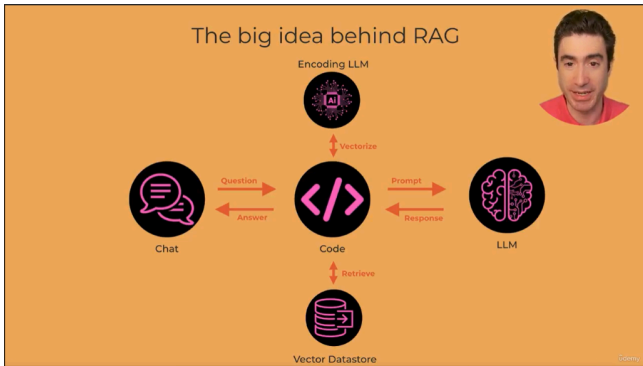
- Các đoạn văn bản có ý nghĩa tương tự sẽ được biểu diễn bằng các vector gần nhau, ngay cả khi chúng sử dụng các từ khác nhau.
- Điều này cho phép tìm kiếm các đoạn văn bản có ý nghĩa tương tự mà không cần so sánh từ ngữ trực tiếp.

Phép toán vector (Vector Math):

- Có thể thực hiện các phép toán trên vector embeddings để khám phá mối quan hệ giữa các khái niệm.
- Ví dụ kinh điển: $\text{vector}(\text{"king"}) - \text{vector}(\text{"man"}) + \text{vector}(\text{"woman"}) \approx \text{vector}(\text{"queen"})$.
- Phép toán này cho thấy rằng vector embeddings thực sự nắm bắt được "ý nghĩa" của các từ và khái niệm.

Ứng dụng của phép toán vector:

- Hiểu được mối quan hệ giữa các khái niệm.
- Tìm kiếm các từ hoặc khái niệm tương tự.
- Thực hiện các tác vụ xử lý ngôn ngữ tự nhiên phức tạp.



Sử dụng Vector Embeddings để tìm kiếm thông tin liên quan:

- Thay vì tìm kiếm thông tin dựa trên từ khóa đơn giản, RAG sử dụng vector embeddings để tìm kiếm thông tin có "ý nghĩa" tương tự với câu hỏi của người dùng.
- Câu hỏi của người dùng được chuyển đổi thành một vector (vectorized).
- Vector này được sử dụng để tìm kiếm các vector tương tự trong cơ sở dữ liệu vector (vector datastore).
- Các tài liệu có vector gần với vector câu hỏi được coi là có ý nghĩa liên quan.

Cơ sở dữ liệu Vector (Vector Datastore):

- Cơ sở dữ liệu vector lưu trữ cả văn bản và vector embeddings tương ứng của văn bản đó.
- Điều này cho phép tìm kiếm thông tin dựa trên ý nghĩa thay vì chỉ từ khóa.

Mô hình ngôn ngữ mã hóa (Encoding LM):

- Mô hình ngôn ngữ mã hóa được sử dụng để chuyển đổi văn bản thành vector embeddings.
- Mô hình này có khả năng nắm bắt ý nghĩa của văn bản và biểu diễn nó dưới dạng một chuỗi số.

Quy trình RAG với Vector Embeddings:

1. Người dùng đặt câu hỏi.
2. Câu hỏi được chuyển đổi thành vector embeddings.
3. Vector embeddings câu hỏi được sử dụng để tìm kiếm các vector tương tự trong cơ sở dữ liệu vector.
4. Các tài liệu liên quan (văn bản) được trích xuất từ cơ sở dữ liệu vector.
5. Văn bản liên quan được thêm vào prompt gửi đến mô hình ngôn ngữ lớn (LLM).
6. LLM tạo ra phản hồi dựa trên prompt đã được bổ sung ngữ cảnh.
7. Phản hồi được gửi lại cho người dùng.

Ưu điểm của phương pháp này:

- Tìm kiếm thông tin chính xác hơn bằng cách xem xét ý nghĩa của văn bản.
- Khắc phục hạn chế của việc tìm kiếm dựa trên từ khóa đơn giản.
- Cải thiện hiệu suất của mô hình ngôn ngữ lớn bằng cách cung cấp ngữ cảnh liên quan.

The complex block features an illustration on the left showing a person walking on a path made of hexagonal tiles towards a distant landscape. On the right is a slide titled 'PROGRESS REPORT RAG Week' with a small portrait of a man. The slide lists tasks under 'What you can now do' and 'By end of next time you'll be able to'.

PROGRESS REPORT RAG Week

What you can now do

- Generate text and code with Frontier Models including AI Assistants with Tools, and with open-source models with HuggingFace transformers
- Confidently choose the right LLM for your project, backed by metrics
- Explain how RAG uses vector embeddings and vector datastores to add context to prompts

By end of next time you'll be able to

- Describe the LangChain framework, with benefits and limitations
- Use LangChain to read in a Knowledge Base of documents
- Use LangChain to divide up documents into overlapping chunks

Tổng kết tuần học về RAG (Retrieval-Augmented Generation):

- Tuần này đã tập trung vào việc giới thiệu và giải thích các khái niệm cơ bản về RAG, đặc biệt là việc sử dụng vector embeddings và cơ sở dữ liệu vector để tìm kiếm thông tin liên quan.
- Bạn đã hiểu được "ý tưởng lớn" đằng sau RAG, đó là sử dụng vector embeddings để tìm kiếm thông tin có "ý nghĩa" tương tự với câu hỏi của người dùng.

Giới thiệu về LangChain:

- Tuần tới, bạn sẽ bắt đầu học về LangChain, một framework mạnh mẽ được thiết kế để đơn giản hóa việc xây dựng các ứng dụng RAG.
- LangChain giúp tự động hóa nhiều tác vụ phức tạp, chẳng hạn như tạo vector embeddings và tương tác với cơ sở dữ liệu vector.
- Nó cho phép bạn thực hiện các tác vụ mạnh mẽ chỉ với một vài dòng code, tương tự như trải nghiệm với Gradio.

Nội dung học tập tuần tới:

- Học cách sử dụng LangChain để triển khai RAG trong thực tế.
- Học cách LangChain đơn giản hóa việc tạo vector embeddings và tương tác với cơ sở dữ liệu vector.
- Học cách xây dựng các ứng dụng RAG mạnh mẽ một cách nhanh chóng và dễ dàng.

